Class Project Guideline

ChiangMai Air Quality Alert System: End-to-End MLOps Pipeline for PM2.5 Forecasting

1. Title & Purpose

Title: ChiangMai Air Quality Alert System: End-to-End MLOps Pipeline for PM2.5 Forecasting

Purpose: To build an automated, production-ready MLOps pipeline that forecasts PM2.5 air pollution levels in Chiang Mai 1, 3, and 7 days in advance — and triggers health alerts before conditions become dangerous. The system is designed to move beyond passive real-time monitoring toward proactive early warning, giving residents and public health agencies time to respond.

2. Project Overview

Problem Statement

Chiang Mai consistently ranks among the most air-polluted cities in the world during its annual haze season (January–April). The root causes are agricultural burning, wildfires in the surrounding highlands, and the city's basin-shaped geography that traps particulate matter close to the ground.

In 2025, the City Hall monitoring station (35T) recorded:

- March average: **55.1 µg/m³** — more than 3× the WHO 24-hour guideline of 15 µg/m³
- Single-day peak: **92.3 µg/m³** (March 23)
- Days exceeding Thailand's safe threshold (>25 µg/m³): **109 days (31% of the year)**

Despite the severity, the current public-facing tools only show real-time AQI. There is no system that:

- Predicts PM2.5 concentration days in advance
- Automatically triggers alerts before levels become hazardous
- Retrains itself as pollution patterns shift over time

Who is affected	Residents, tourists, public health agencies, city planners
Core problem	No predictive early warning system exists — only real-time monitoring
Impact	Preventable health exposure due to lack of advance notice
Gap	Tools show what is happening now; none forecast what is coming
Our solution	End-to-end MLOps pipeline: daily forecast (t+1/3/7 days) + automated alert + drift-aware retraining

Project Type

End-to-End Data Engineering + MLOps

This project covers the full lifecycle: multi-source data ingestion → feature engineering → model training and evaluation → deployment → real-time serving → monitoring and automated retraining.

---

3. Data Pipeline Design

Data Sources

Source	Provider	Type	Coverage	Granularity
PM2.5 concentration	air4thai / PCD Thailand	File (Excel) + API	2011–2025, stations 35T & 36T, Chiang Mai	Daily average
Historical weather	Open-Meteo API	API (REST)	Chiang Mai coordinates	Daily
Fire hotspots	NASA FIRMS API	API (REST)	Northern Thailand bounding box	Daily

Primary station: 35T — Chiang Mai City Hall, Chang Phueak **Backup station:** 36T — Yupparaj School, Si Phum (Pearson r = 0.94 with 35T) **Training data size:** ~5,475 rows (15 years × 365 days)

Ingestion Method

- **Historical load (one-time):** Bulk parse from PCD Excel files (2011–2025), one file per year
- **Ongoing ingestion (daily batch):** Scheduled pipeline runs at 06:00 every morning — pulls previous day's PM2.5 via air4thai API, weather via Open-Meteo, and fire hotspots via NASA FIRMS
- **Method:** Batch ingestion — real-time streaming is not required for a daily forecast use case

Storage Layers

```
[Raw Layer]
    S3 bucket: raw/pm25/YYYY-MM-DD.csv
               raw/weather/YYYY-MM-DD.csv
               raw/fire/YYYY-MM-DD.csv
    → Immutable, append-only, never overwritten

[Processed Layer]
    S3 bucket: processed/merged/YYYY-MM-DD.csv
    → Cleaned, validated, schema-enforced, missing values handled

[Feature Store]
    S3 bucket: features/pm25_daily_chiangmai.csv
    → ML-ready: all features + target columns, versioned by run date

[Model Artifacts]
    MLflow artifact store (S3 or local)
    → Trained model files, metrics, plots per experiment run
```

```
[Serving Layer]
    FastAPI (in-memory model) → /predict, /alert endpoints
    Forecast results stored back to S3: predictions/YYYY-MM-DD.csv
```

Transformations & Orchestration

Transformation steps (in order):

1. Schema validation — check expected columns, data types, date range
2. Missing value handling — interpolate short gaps (≤3 days); use 36T as fallback for PM2.5; fill fire count = 0 on days with no hotspots
3. Unit normalization — standardize column names, numeric types, date format
4. Feature engineering — lag, rolling, cyclical encoding, season flags, regional aggregates (see Section 4.1)
5. Target variable creation — shift PM2.5 forward by 1, 3, 7 days

Orchestration:

- GitHub Actions schedules the daily pipeline (cron: `0 6 * * *`)
- Each step logs success/failure explicitly — pipeline halts on any error, no silent failures
- Checkpoints saved after each step to allow resume without re-running from scratch

---

4. MLOps Lifecycle

4.1 Feature Engineering

Features are grouped into three categories:

Temporal features

Feature	Description
`PM25_lag1/2/3/7/14`	PM2.5 value 1–14 days ago — captures short-term autocorrelation
`PM25_roll3/7/14/30_mean`	Rolling average over 3–30 days — captures medium-term trends
`PM25_roll7/30_std`	Rolling standard deviation — captures volatility
`month_sin`, `month_cos`	Cyclical encoding of month — prevents ordinality assumptions
`day_sin`, `day_cos`	Cyclical encoding of day-of-year

Contextual features

Feature	Description
`is_haze_season`	1 if January–April
`is_rainy_season`	1 if May–October
`north_avg_PM25`	Average PM2.5 across 15 northern Thailand stations — regional smoke signal

Feature	Description
`north_max_PM25`	Max PM2.5 across northern stations
`wind_speed`, `wind_dir`	Wind from Open-Meteo — affects smoke transport
`humidity`, `temperature`	From Open-Meteo
`boundary_layer_height`	Atmospheric mixing height — key driver of surface PM2.5 accumulation
`fire_count`	Daily hotspot count within 100 km of Chiang Mai (NASA FIRMS)
`fire_frp_sum`	Total Fire Radiative Power — proxy for fire intensity

Target variables

Variable	Type	Definition
`PM25_next1`	Regression	PM2.5 tomorrow (µg/m³)
`PM25_next3`	Regression	PM2.5 in 3 days
`PM25_next7`	Regression	PM2.5 in 7 days
`alert_next1`	Binary classification	1 if PM2.5 tomorrow > 50 µg/m³
`alert_next3`	Binary classification	1 if PM2.5 in 3 days > 50 µg/m³

4.2 Model Training & Evaluation

Models:

XGBoost (baseline)

- Input: full tabular feature set
- Purpose: establish a strong baseline and validate feature importance before building sequential models
- Advantages: fast training, native missing value handling, interpretable via SHAP

LSTM (primary model)

- Input: 30-day rolling sequence window
- Output: PM2.5 forecast for t+1, t+3, or t+7
- Rationale: captures long-range seasonal dependencies and multi-day momentum that tabular models cannot express

Train/Validation/Test split (chronological — no shuffling):

Split	Period	Rationale
Train	2011–2022	12 years, multiple haze seasons
Validation	2023	Hyperparameter tuning
Test	2024–2025	Fully unseen, recent haze seasons included

Evaluation metrics:

Task	Metrics	Priority
Regression	RMSE, MAE, R^2	RMSE primary
Classification	Precision, Recall, F1	**Recall primary** — missing a true alert is more harmful than a false alarm

Baseline to beat: $R^2 = 0.70$ (reference benchmark from similar regional PM2.5 forecasting work)

4.3 Experiment Tracking

Tool: MLflow (self-hosted or via MLflow on AWS)

Every training run logs:

- Hyperparameters: learning rate, n_estimators, LSTM units, sequence length, dropout rate
- Metrics: RMSE, MAE, R^2, F1, Recall per fold and overall
- Artifacts: trained model file, feature importance plot, confusion matrix, loss curves
- Dataset version: date of training data snapshot used

This enables full reproducibility and direct side-by-side comparison between XGBoost and LSTM runs across experiments.

4.4 Model Versioning

Models are managed through **MLflow Model Registry** with three promotion stages:

```
[Staging] → [Validation] → [Production]
```

- **Staging:** Newly trained model, awaiting metric evaluation
- **Validation:** Passed RMSE and Recall thresholds, ready for deployment review
- **Production:** Currently serving model — only one model at this stage at any time

Promotion to Production is automated when metrics pass. Previous Production models are archived (not deleted) to allow rollback if needed.

4.5 Deployment Strategy

Hybrid: Batch + Real-Time API

Batch pipeline (primary):

- Runs daily at 06:00 via GitHub Actions
- Ingests previous day's data → generates t+1/3/7 forecasts → stores results to S3
- Appropriate for a daily alert use case — no sub-second latency required
- Cost-efficient: scheduled Lambda or lightweight EC2

Real-time API (for dashboard and integrations):

- FastAPI serving `/predict` and `/alert` endpoints
- Model loaded into memory at startup
- Used by the Streamlit dashboard and any external consumers
- Deployed on AWS EC2 t3.micro or App Runner (auto-scales to zero)

4.6 Monitoring & Retraining Plan

Tools: Evidently AI (drift detection) + MLflow (performance tracking) + GitHub Actions (CI/CD trigger)

Data drift monitoring:

- Weekly Evidently report comparing current feature distributions vs. training baseline
- Key features monitored: `PM25_lag1`, `north_avg_PM25`, `wind_speed`, `fire_count`
- Statistical tests: PSI (Population Stability Index) + KS-test
- Drift score exceeds threshold → flag for team review

Model performance monitoring:

- After each daily batch run, yesterday's prediction is compared against actual observed PM2.5
- 7-day rolling RMSE and MAE logged continuously to MLflow
- If rolling RMSE increases >20% above training baseline → automated retraining triggered

Alert accuracy tracking:

- False negative rate (missed alerts) monitored over 30-day windows
- If false negative rate > 15% → immediate retraining regardless of RMSE

Automated retraining pipeline:

```
Drift detected OR RMSE threshold exceeded
    → GitHub Actions triggers training job
    → New model logged in MLflow
    → If metrics pass threshold → promote to Production automatically
    → If metrics fail → notify team for manual review
```

Scheduled retraining: Minimum once per year in December, before haze season, incorporating the most recent full year of data.

---

5. Infrastructure & Cost Awareness

AWS Architecture (AWS Learner Lab)

```
[Ingestion & Storage]
    S3  →  Raw + processed data storage, model artifacts, prediction
results

[Compute]
    Lambda (scheduled)  →  Daily batch pipeline (ingest + predict)
```

```
    EC2 t3.micro        →  FastAPI model serving (always-on)

[Orchestration]
    GitHub Actions  →  CI/CD, pipeline scheduling, retraining trigger

[Monitoring]
    Evidently AI    →  Drift reports (runs inside Lambda or EC2)
    MLflow          →  Experiment tracking + model registry (EC2 or local)

[Dashboard]
    Streamlit       →  Deployed on EC2 or Streamlit Community Cloud (free)
```

Major Cost Drivers

Service	Usage	Estimated Monthly Cost
S3 storage	~1–2 GB data + artifacts	~$0.03
Lambda	Daily batch run, ~5 min/day	~$0.00 (within free tier)
EC2 t3.micro	API serving (always-on)	~$7–8 (or $0 with Learner Lab credits)
Data transfer	API calls to Open-Meteo, FIRMS	~$0.00 (both free APIs)
Total		**~$2–8/month**

> Reference project with similar scope: ~$2.76/month. Our estimate is in the same range, especially with Learner Lab credits covering EC2.

Trade-offs Considered

- **Lambda vs EC2 for serving:** Lambda is cheaper for low-traffic but has cold start latency (~1–2s). EC2 t3.micro provides consistent response time for the dashboard. Chose EC2 for serving, Lambda for batch only.
- **Self-hosted MLflow vs managed:** Self-hosted on EC2 avoids additional managed service cost. Acceptable for a project-scale deployment.
- **Streamlit Community Cloud:** Free option for dashboard hosting — avoids EC2 cost for the frontend entirely if traffic is low.

---

6. Data Quality & Monitoring

6.1 Data Quality

Missing values:

Source	Expected missing rate	Handling strategy

Source	Expected missing rate	Handling strategy
air4thai PM2.5	~2–3% per year	If ≤3 consecutive days: interpolate from lag + rolling mean. If longer: use 36T station (r = 0.94) as substitute. Flag all imputed rows with `is_imputed = 1` column.
Open-Meteo weather	Near zero (reanalysis)	No action required
NASA FIRMS fire	Zero hotspots on some days is valid (not missing)	Fill `fire_count = 0`, `fire_frp_sum = 0`

Schema validation:

- Earlier PCD files (2011–2017) may differ in column count and station coverage due to network expansion over time
- Every pipeline run validates: expected columns present, correct data types, date range within bounds, no duplicate dates
- If validation fails → pipeline halts immediately and raises an alert. No silent failures allowed.

6.2 Pipeline Health

Failure monitoring:

- Each pipeline step (ingest → preprocess → feature engineering → predict) produces structured logs with step name, timestamp, row counts, and status
- If any step fails, the pipeline stops immediately — downstream steps never run on incomplete data
- GitHub Actions sends email notification on any pipeline failure

Latency monitoring:

- Daily batch pipeline timeout: 30 minutes
- Expected normal runtime: 5–10 minutes
- Timeout breach triggers an alert for investigation

Data freshness check:

- After each ingestion run, system verifies that the latest PM2.5 record date matches yesterday
- If data is more than 2 days stale → source flagged as potentially unavailable, alert raised

6.3 Model Performance Monitoring

- After each daily batch run, yesterday's PM2.5 prediction is compared against the actual observed value (now known)
- 7-day rolling RMSE and MAE computed and logged to MLflow
- Retraining triggered automatically if rolling RMSE exceeds 120% of training baseline
- All historical metrics retained in MLflow for long-term trend analysis
- Alert false negative rate monitored separately over 30-day windows (threshold: 15%)

6.4 Data Drift & Performance Degradation

Data drift (Evidently AI):

- Weekly distribution comparison: current week's features vs. training data baseline
- Priority features: `PM25_lag1`, `north_avg_PM25`, `wind_speed`, `fire_count`
- Statistical tests: PSI and KS-test
- Drift score breach → flagged for team review before retraining decision

Concept drift:

- PM2.5 seasonal patterns may shift year-over-year (e.g., haze season starting earlier, increasing wildfire frequency due to climate change)
- Mitigation: annual retraining in December, incorporating the most recent full year of data so the model reflects current environmental patterns

---

7. Team Progress

Roles & Task Ownership

Member	Role	Responsibilities
Supanut	ML & Data Engineering	Data pipeline, feature engineering, model training (XGBoost + LSTM), experiment tracking (MLflow), FastAPI, Streamlit dashboard
Shuvam	Infrastructure & DevOps	AWS architecture design, cost estimation, CI/CD pipeline (GitHub Actions), Evidently AI monitoring setup